

ShopSphere: Abschlussprojekt

Block 1 — SQL: Datenvorbereitung

Zusatzdatei: Schritt-für-Schritt-Analyse der Syntax jeder Abfrage

Dieses Dokument ist mein Arbeitsjournal als Analyst. Für jede der 5 Abfragen aus Block 1 zeige ich: (1) den SQL-Text selbst, (2) eine Schritt-für-Schritt-Analyse – was jeder Teil der Abfrage bewirkt und warum, (3) das tatsächliche Ausführungsergebnis anhand der ShopSphere-Daten, (4) ein kurzes Business-Fazit. Dieses Format hilft dabei, nicht nur „eine funktionierende Abfrage zu schreiben“, sondern die Logik jeder Zeile zu verstehen – genau das unterscheidet jemanden, der nur Syntax kopiert, von einem Analysten.

1.1. Umsatz, Bestellanzahl und durchschnittlicher Bestellwert nach Region und Jahr

Ziel: zeigen, wie das Geschäft nach Region × Jahr wächst – die Grundlage für Grafik 2.4 (Regionsentwicklung) und die KPI-Karten des Dashboards.

SQL-Abfrage

```
SELECT c.region,
       o.order_year,
       ROUND(SUM(o.net_amount), 2) AS total_net_revenue,
       COUNT(o.order_id) AS order_count,
       ROUND(AVG(o.net_amount), 2) AS avg_order_value
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.region, o.order_year
ORDER BY c.region, o.order_year;
```

Schritt-für-Schritt-Syntaxanalyse

- **FROM orders o** – wir beginnen mit der Tabelle orders (Bestellungen) – sie ist die Haupttabelle, da hier der Umsatz (net_amount) liegt. Ich vergebe den kurzen Alias „o“, um nicht in jeder Spalte den vollen Namen schreiben zu müssen.
- **JOIN customers c ON o.customer_id = c.customer_id** – die Tabelle orders enthält keine Spalte region – diese gibt es nur in customers. Deshalb „verbinde“ (JOIN) ich die Tabelle customers über den gemeinsamen Schlüssel customer_id. ON gibt die Zuordnungsregel an: Eine Bestellzeile wird genau mit dem Kunden verknüpft, dessen customer_id übereinstimmt.
- **SELECT c.region, o.order_year, ...** – hier liste ich auf, welche Spalten ich im Ergebnis sehen möchte: Region (aus customers), Jahr (aus orders) und drei berechnete Kennzahlen.
- **SUM(o.net_amount)** – eine Aggregatfunktion – summiert net_amount über alle Zeilen innerhalb jeder Gruppe (Region+Jahr). Ohne GROUP BY würde sie einfach eine einzige Gesamtsumme über die gesamte Tabelle liefern.
- **ROUND(... , 2)** – rundet eine Dezimalzahl auf 2 Nachkommastellen – damit Dollarbeträge nicht wie 100827.259999 aussehen, sondern wie 100827,26.
- **COUNT(o.order_id)** – zählt die Anzahl der Zeilen (Bestellungen) in jeder Gruppe – das ist die „Anzahl der Bestellungen“.
- **AVG(o.net_amount)** – berechnet den arithmetischen Mittelwert von net_amount in der Gruppe – den durchschnittlichen Bestellwert.
- **AS total_net_revenue / AS order_count / ...** – AS benennt die Ergebnisspalte um (Alias), damit der Name in der CSV-Datei und später in Tableau verständlich ist, statt „SUM(o.net_amount)“ zu heißen.
- **GROUP BY c.region, o.order_year** – die wichtigste Zeile: Sie weist die Datenbank an, „alle Zeilen mit derselben Region UND demselben Jahr zu einer Gruppe zusammenzufassen und die

Aggregate (SUM/COUNT/AVG) für jede dieser Gruppen separat zu berechnen“. Ohne GROUP BY funktionieren Aggregatfunktionen nicht zusammen mit normalen Spalten.

- **ORDER BY c.region, o.order_year** – sortiert das Endergebnis zunächst nach Region (alphabetisch) und innerhalb der Region nach Jahr aufsteigend, damit die Zeilen logisch geordnet sind (2022 → 2023 → 2024).

Ergebnis (Ausschnitt)

region	order_year	total_net_revenue	order_count	avg_order_value
North America	2022	80 204,08	254	315,76
North America	2023	235 736,97	848	277,99
North America	2024	718 726,68	2632	273,07
Southeast Asia	2022	12 721,48	44	289,12
Southeast Asia	2024	613 904,12	2029	302,56

Business-Fazit: Alle 5 Regionen wachsen von Jahr zu Jahr, jedoch mit unterschiedlicher Geschwindigkeit. Southeast Asia und Latin America wachsen relativ am schnellsten (ausgehend von einer sehr kleinen Basis), während North America und Europe in absoluten Umsatzzahlen am größten bleiben. Die vollständige Tabelle ist in der Datei 1_1_region_year.csv für Tableau gespeichert.

1.2. Top-10-Kunden nach Gesamtausgaben

Ziel: die wertvollsten Kunden für Frage 9 (Top 5 %) identifizieren und verstehen, woher sie kommen (Akquisitionskanal) – Grundlage für die Bindung von VIP-Kunden.

SQL-Abfrage

```
SELECT o.customer_id,
       c.region,
       c.acquisition_channel,
       COUNT(o.order_id) AS orders_count,
       ROUND(SUM(o.net_amount),2) AS total_spent
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY o.customer_id, c.region, c.acquisition_channel
ORDER BY total_spent DESC
LIMIT 10;
```

Schritt-für-Schritt-Syntaxanalyse

- **GROUP BY o.customer_id, c.region, c.acquisition_channel** – ich gruppiere gleichzeitig nach drei Spalten. Da region und acquisition_channel für jede customer_id eindeutig sind (ein Kunde – eine Region), handelt es sich faktisch um eine Gruppierung „pro Kunde“, aber SQL verlangt, dass alle nicht aggregierten Spalten im SELECT auch in GROUP BY stehen.
- **ORDER BY total_spent DESC** – sortiert absteigend (DESC = descending) – die höchsten Ausgaben stehen oben. Ohne DESC würde aufsteigend sortiert (die niedrigsten oben) – nicht das, was für ein „Top“ gebraucht wird.
- **LIMIT 10** – behält nur die ersten 10 Zeilen nach der Sortierung – so erhält man genau die „Top 10“ und nicht alle Kunden.

Ergebnis (alle 10)

customer_id	region	acquisition_channel	orders_count	total_spent
12348	Europe	Influencer	43	30 360,93
11131	Europe	Influencer	41	22 546,65
12723	North America	Referral	43	19 561,76
10995	Europe	Influencer	32	19 496,63
11387	Southeast Asia	Influencer	35	18 730,10
11835	Southeast Asia	Referral	31	18 114,42
11046	Europe	Organic	37	17 960,45
10837	North America	Influencer	27	17 822,03
12965	Europe	Email	32	16 732,93
12478	North America	Referral	32	16 412,50

Business-Fazit: 6 der 10 wertvollsten Kunden kamen über den Influencer-Kanal – ein starkes Signal für Frage 4 (LTV nach Akquisitionskanal).

1.3. Umsatz, Marge und Rücksendeanteil nach Kategorien

Ziel: erkennen, welche Kategorie Volumen liefert und welche echten Gewinn bringt. Grundlage für das Streudiagramm 2.3 und Case B (Block 4).

SQL-Abfrage (mit CTE – einer temporären „Untertabelle“)

```
WITH cat_orders AS (
  SELECT DISTINCT oi.category, oi.order_id, o.is_returned
  FROM order_items oi
  JOIN orders o ON oi.order_id = o.order_id
)
SELECT oi.category,
       ROUND(SUM(oi.line_total), 2) AS total_revenue,
       ROUND(AVG(p.margin_pct), 2) AS avg_margin_pct,
       (SELECT ROUND(100.0 * SUM(is_returned) / COUNT(*), 2)
        FROM cat_orders co WHERE co.category = oi.category) AS return_rate_pct
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY oi.category
ORDER BY total_revenue DESC;
```

Schritt-für-Schritt-Syntaxanalyse

- **WITH cat_orders AS (...)** – dies ist ein CTE (Common Table Expression) – eine „temporäre benannte Tabelle“, die nur während der Ausführung dieser Abfrage existiert. Sie wird benötigt, weil eine Bestellung mehrere Produkte enthalten kann (mehrere Zeilen in order_items), und wenn man den Rücksendeanteil direkt über order_items berechnen würde, würde eine zurückgesendete Bestellung mehrfach gezählt (einmal pro enthaltenem Produkt). Der CTE führt zunächst ein DISTINCT über das Paar (Kategorie, order_id) aus, damit jede Bestellung in einer Kategorie genau einmal gezählt wird.
- **SELECT DISTINCT oi.category, oi.order_id, o.is_returned** – DISTINCT entfernt doppelte Zeilen – es bleiben nur eindeutige Kombinationen aus Kategorie+Bestellung, selbst wenn es in order_items mehrere solcher Zeilen gab.

- **(SELECT ... FROM cat_orders co WHERE co.category = oi.category)** – dies ist eine korrelierte Unterabfrage (correlated subquery) – ein kleines SELECT, das direkt in die Spaltenliste der Hauptabfrage eingebettet ist. Es wird für jede Kategorie aus der äußeren Abfrage separat ausgeführt (daher WHERE co.category = oi.category – die „Verbindung“ zur äußeren Zeile) und berechnet den Rücksendeanteil genau für diese Kategorie.
- **JOIN products p ON oi.product_id = p.product_id** – ich verbinde products, um margin_pct zu erhalten – diese Spalte gibt es weder in order_items noch in orders.
- **AVG(p.margin_pct)** – die durchschnittliche Marge über alle Produktzeilen der Kategorie (nicht über eindeutige Produkte, sondern gewichtet nach der Anzahl verkaufter Einheiten/Zeilen) – zeigt die „typische“ Profitabilität der Kategorie.

Ergebnis (alle 7 Kategorien)

category	total_revenue	avg_margin_pct	return_rate_pct
Electronics	2 097 901,06	12,0	15,97
Home & Kitchen	576 134,75	35,0	10,27
Sports	343 114,98	30,0	8,40
Clothing	248 601,48	45,0	16,00
Beauty	168 624,42	55,0	9,97
Toys	140 505,55	40,0	8,98
Books	90 757,82	25,0	8,13

Business-Fazit: Electronics erwirtschaftet 60 % des gesamten Umsatzes, hat aber unter den großen Kategorien die niedrigste Marge (12 %) und den höchsten Rücksendeanteil (16 %) – eine klassische „Volumen-Illusion“ (siehe Frage 6). Beauty ist das Gegenteil: geringer Umsatz, aber 55 % Marge und niedrige Rücksendequote – ein „verborgener Diamant“ (Frage 7).

1.4. Kunden mit überdurchschnittlichen Ausgaben (Unterabfrage)

Ziel: prüfen, wie stark der Unternehmensumsatz von einer Minderheit an Kunden abhängt, die mehr ausgeben als der „typische“ Kunde.

SQL-Abfrage

```
WITH customer_totals AS (
    SELECT
        customer_id,
        SUM(net_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
),
avg_spending AS (
    SELECT AVG(total_spent) AS avg_total_spent
    FROM customer_totals
),
```

```

above_avg AS (
    SELECT *
    FROM customer_totals ct
    CROSS JOIN avg_spending a
    WHERE ct.total_spent > a.avg_total_spent
)

SELECT
    COUNT(*) AS above_avg_customers,
    ROUND(SUM(total_spent), 2) AS above_avg_total_revenue,

    -- Gesamtumsatz aller Kunden -- -- Загальний обсяг продажів усіх клієнтів
    (SELECT SUM(total_spent) FROM customer_totals) AS grand_total_revenue,

    -- Umsatzanteil -- Частка у виручці
    ROUND(
        SUM(total_spent) * 100.0 /
        (SELECT SUM(total_spent) FROM customer_totals)
    , 2) AS share_of_revenue_pct,

    -- Gesamtzahl der Kunden -- -- Загальна кількість клієнтів
    (SELECT COUNT(*) FROM customer_totals) AS total_customers,

    -- Kundenanteil -- Частка клієнтів
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM customer_totals)
    , 2) AS share_of_customers_pct
FROM above_avg;

```

Schritt-für-Schritt-Syntaxanalyse

- **FROM (SELECT customer_id, SUM(net_amount) ...) t** – dies ist eine Unterabfrage in FROM („abgeleitete Tabelle“): Zunächst wird die Gesamtausgabe JEDES einzelnen Kunden berechnet, und dieses Zwischenergebnis wird „t“ genannt, als wäre es eine gewöhnliche Tabelle.
- **WHERE t.total_spent > (SELECT AVG(total_spent) FROM (...))** – dies ist eine skalare Unterabfrage (liefert eine einzige Zahl) – sie berechnet die durchschnittliche Ausgabe über ALLE Kunden. WHERE lässt dann in „t“ nur die Kunden übrig, deren Ausgabensumme über diesem Durchschnitt liegt.

- **Warum zwei verschachtelte, ähnliche Unterabfragen?** – die erste (in FROM) bereitet die „Ausgaben pro Kunde“ vor, die zweite (in WHERE) berechnet den Durchschnitt aus derselben Zwischentabelle. SQL erlaubt es nicht, eine Zeile direkt mit einem Aggregat im selben SELECT ohne Gruppierung zu vergleichen – eine Unterabfrage ist der Standardweg, dies zu umgehen.
- **COUNT(*) / SUM(total_spent)** – in der äußeren Abfrage wird gezählt, wie viele solcher Kunden es gibt (above_avg_customers) und welchen Gesamtumsatz sie erwirtschaften.

Ergebnis

above_avg_customers	above_avg_total_revenue	grand_total_revenue	share_of_revenue_pct	total_customers	share_of_revenue_pct
862	2 524 753,93	3 474 016,03	72,68	3000	28,7

Business-Fazit: Nur 28,7 % der Kunden (862 von 3000) geben überdurchschnittlich viel aus, erwirtschaften jedoch 72,7 % des gesamten Umsatzes. Das bestätigt eine starke Wertkonzentration (Pareto-Prinzip) und bereitet unmittelbar den Boden für Frage 9 (Top-5%-Kunden).

1.5. ROI der Marketingkanäle

Ziel: berechnen, welcher Marketingkanal das Budget effizienter in Umsatz umwandelt – Schlüsseldaten für Case A (Frage 3).

SQL-Abfrage

```
SELECT channel,
       ROUND(SUM(budget), 2) AS total_budget,
       ROUND(SUM(attributed_revenue), 2) AS total_attributed_revenue,
       ROUND(SUM(attributed_revenue) / SUM(budget), 2) AS roi
FROM marketing
GROUP BY channel
ORDER BY roi DESC;
```

Schritt-für-Schritt-Syntaxanalyse

- **SUM(budget) / SUM(attributed_revenue)** – wir aggregieren (summieren) Budget und Umsatz getrennt über alle 216 Kampagnen-Einträge (jeder Eintrag = Monat × Kanal), statt den ROI für jeden Monat einzeln zu berechnen und dann zu mitteln – das ist wichtig, weil eine Mittelung des ROI über Monate ein verzerrtes Ergebnis liefern würde, wenn sich die monatlichen Budgets stark unterscheiden.
- **SUM(attributed_revenue) / SUM(budget) AS roi** – ROI (Return on Investment) ist hier definiert als „wie viele Dollar Umsatz pro investiertem Dollar Budget zurückkommen“. Ein Wert von 1,0 bedeutet „bei null herausgekommen“, über 1,0 ist profitabel, darunter ein Verlust im Verhältnis zu den Werbeausgaben.
- **GROUP BY channel** – wir gruppieren genau nach Kanal (Organic, Paid Search, Social Ads, Influencer, Email, Referral) und ignorieren Jahr und Monat – wir brauchen das Gesamtbild über den gesamten Zeitraum 2022–2024.
- **ORDER BY roi DESC** – wir sortieren vom effizientesten zum am wenigsten effizienten Kanal – so sind Spitzenreiter und Schlusslicht sofort erkennbar.

Ergebnis (alle 6 Kanäle)

channel	total_budget	total_attributed_revenue	roi
Organic	20 364,00	163 398,00	8,0
Email	37 468,00	243 610,00	6,0

channel	total_budget	total_attributed_revenue	roi
Influencer	112 337,00	519 453,00	4,0
Referral	73 766,00	263 536,00	3,0
Social Ads	286 488,00	589 544,00	2,0
Paid Search	450 959,00	598 703,00	1,0

Business-Fazit: Paid Search erhält das größte Budget (450 959 \$ – 45 % aller Marketingausgaben), hat aber den niedrigsten ROI (1,0). Organic und Email sind am effizientesten, erhalten aber nur einen Bruchteil des Budgets. Eine detaillierte Analyse und Empfehlung folgt in den Fragen 3–5.